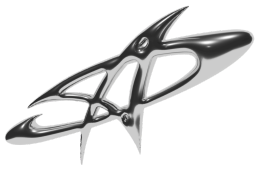
Week 04 · Multi-Head Attention & Positional Encoding

여러 관점으로 동시에 보고, 순서를 기억하게 만들기

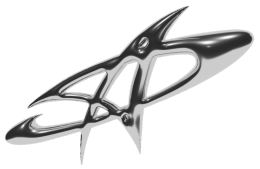
Attention을 여러 head로 나눠 병렬로 수행하는 이유, 그리고 Attention이 놓치는 “순서” 정보를 Positional Encoding으로 채워 넣는 방법을 다룹니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

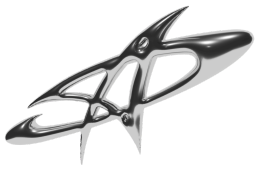
왜 Head를 여러 개로 나누어야 하는가



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

우리가 3주차에 만든 Attention은 딱 한 가지 관점으로만 단어 사이 관계를 본다.

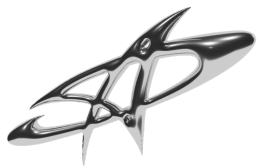
하지만 실제 언어는 동시에 여러 관점을 필요로 한다 —
문법적 관계, 의미적 유사성, 지시어 관계 등을 한꺼번에 봐야 한다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

하나의 큰 Attention 대신, 작은 Attention을 h개 병렬로

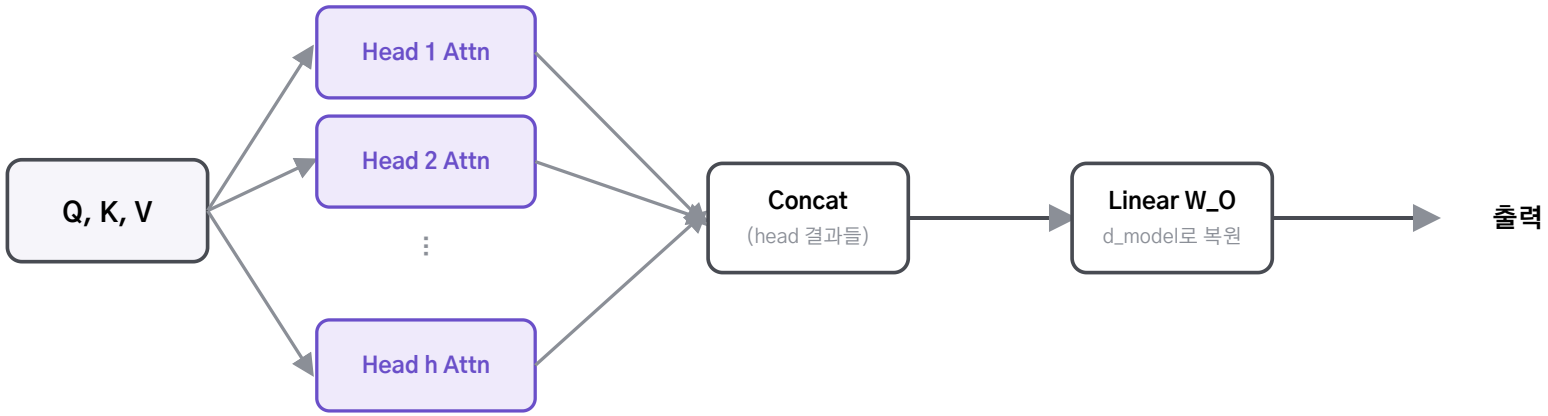
- d_model 차원을 **h개의 head**로 나눠, 각 head가 d_k = d_model/h 차원에서 독립적으로 Attention 수행
- 예: d_model=512, h=8 → 각 head는 64차원에서 Attention 계산
- 각 head는 **서로 다른 W_Q, W_K, W_V**를 가지므로, 학습이 진행되며 서로 다른 관점(예: 문법 관계 vs 의미 관계)을 익히게 됨
- 모든 head의 결과를 **이어붙인(concat) 뒤**, 선형변환(W_O) 하나로 다시 d_model 차원으로 합침



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Multi-Head Attention 구조

나누고(Split) → 각자 계산 → 합치고(Concat)



각 head가 자기 몫(d_k)에서만 계산하므로, 전체 연산량은 “head 1개짜리 풀사이즈 Attention”과 거의 같다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Multi-Head Attention 수식

head를 계산하고, 이어붙이고, 한 번 더 투영

각 head는 이렇게 계산됩니다: head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)

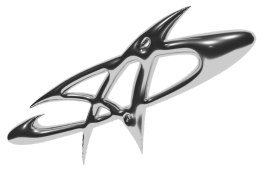
$$\mathrm{MultiHead}(Q,K,V) = \mathrm{Concat}(\mathrm{head}_1,\ldots,\mathrm{head}_h)\,W^O$$

h — head의 개수 (원 논문 기준 8)

d_k = d_model / h — head 하나가 다루는 차원

W_i^Q, W_i^K, W_i^V — head마다 독립적으로 학습되는 투영 행렬

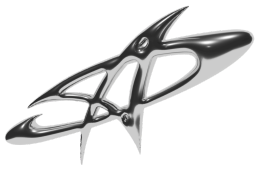
W^O — concat된 결과를 다시 d_model로 되돌리는 출력 투영



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Self-Attention은 모든 단어를 동시에, 대칭적으로 처리한다.
단어 순서를 뒤섞어도 Attention 계산 결과는 수학적으로 동일하다.

즉, Attention은 그 자체로는 “몇 번째 단어인지”를 전혀 모른다.

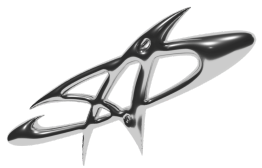


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Positional Encoding이 필요한 이유

“순서 없는 집합”에 순서를 새겨 넣는다

- RNN은 구조 자체가 순차적이라 순서를 자연히 반영했지만, Attention은 그렇지 않음 (순서-불변, permutation invariant)
- 해결책: 각 단어의 **위치 정보를 벡터로 만들어**, 입력 임베딩에 **더해준다(add)**
- 단순히 위치 번호(0,1,2,3…)를 그대로 쓰지 않는 이유: 문장이 길어지면 값이 너무 커지고, 상대적 거리 정보를 표현하기 어려움
- 대신 **여러 주파수의 sin/cos 파형**을 사용 — 학습 없이도 일정한 규칙을 갖는 위치 벡터를 만들 수 있음



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Positional Encoding 수식

서로 다른 주파수의 sin / cos

$$PE_{(pos,2i)} = \sin\left(pos \;/\; 10000^{2i/d_{model}} \right)$$

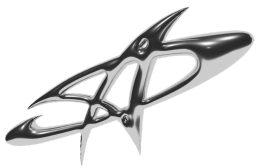
$$PE_{(pos,2i+1)} = \cos\left(pos \;/\; 10000^{2i/d_{model}} \right)$$

pos — 문장 안에서 단어의 위치 (0, 1, 2, ⋯)

i — 임베딩 벡터 내의 차원 인덱스

짝수 차원엔 **sin**, 홀수 차원엔 **cos** — 같은 위치라도 차원마다 다른 주파수의 파형 값을 가짐

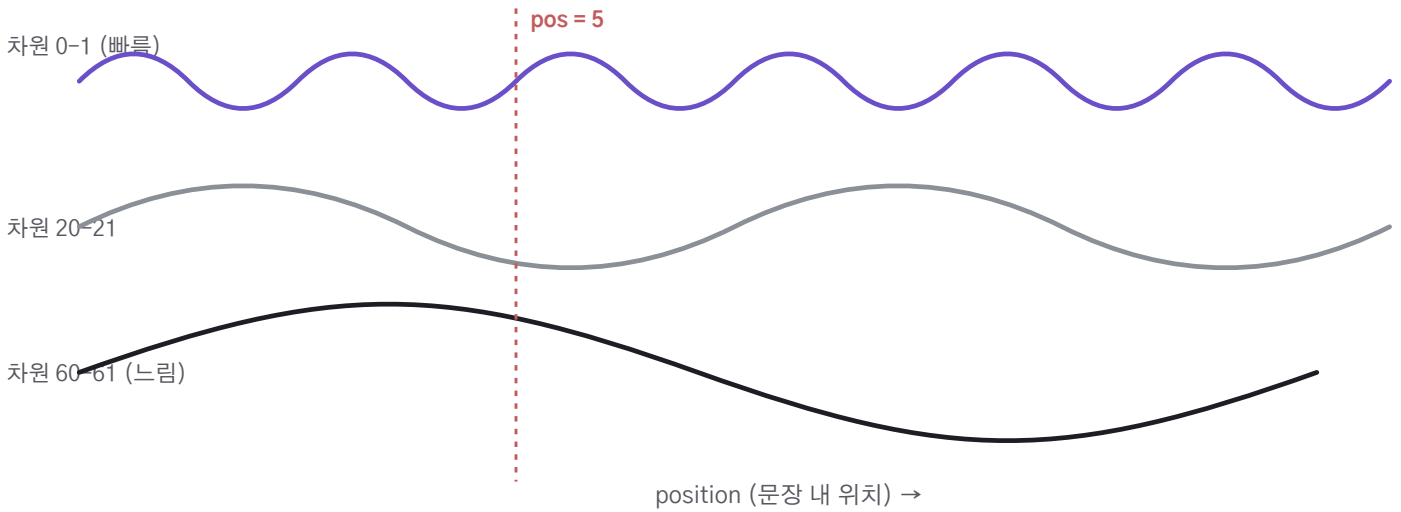
장점 — 학습 없이 정의되고, 학습 때 못 본 더 긴 문장에도 자연스럽게 확장 가능



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

실제 파형을 보면

차원마다 다른 속도로 진동한다



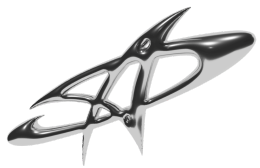
낮은 차원은 빠르게, 높은 차원은 느리게 진동합니다 — 같은 위치(pos=5)라도 차원마다 값이 달라 “그 위치만의 고유한 지문(fingerprint)”이 만들어집니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

왜 “더하기(add)”인가

Concat이 아니라 Add를 쓰는 이유

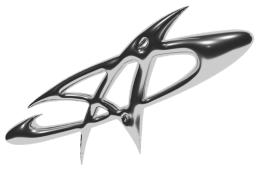
- PE를 임베딩과 **같은 차원(d_model)**으로 만들어 **그대로 더함** — 차원이 늘어나지 않아 연산량이 그대로 유지됨
- 모델은 학습 과정에서 “더해진 값” 안에서 내용 정보와 위치 정보를 스스로 분리해 활용하는 법을 익힘
- 이 강의에서는 원 논문 그대로 **고정된 sin/cos PE**를 사용 (BERT 등 일부 모델은 학습되는 위치 임베딩을 쓰기도 함 — 선택의 문제)



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

MultiHeadAttention 클래스 + Positional Encoding 구현



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

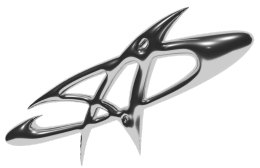
d_model을 h개의 head로 나누기

01_mha_init.py

```python
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.h = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def split_heads(self, x):
        B, T, _ = x.shape
        x = x.view(B, T, self.h, self.d_k)
        return x.transpose(1, 2)   # [B, h, T, d_k]
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

MultiHeadAttention (2) — forward

attention → concat → 출력 투영

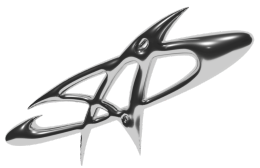
02_mha_forward.py

```
    def forward(self, x, mask=None):
        B, T, _ = x.shape
        Q = self.split_heads(self.W_q(x))   # [B, h, T, d_k]
        K = self.split_heads(self.W_k(x))
        V = self.split_heads(self.W_v(x))

        out, weights = scaled_dot_product_attention(Q, K, V, mask)
        # out: [B, h, T, d_k] → 다시 [B, T, d_model]로 합치기
        out = out.transpose(1, 2).contiguous().view(B, T, -1)

        return self.W_o(out)
```

3주차에 만든 **scaled_dot_product_attention** 함수를 그대로 재사용합니다 — head 차원(h)이 배치처럼 취급되어 병렬로 계산됩니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Positional Encoding 구현

sin/cos 행렬을 미리 만들어두기

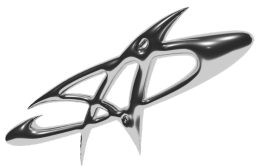
03_positional_encoding.py

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=512):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer("pe", pe.unsqueeze(0)) # [1, max_len, d_model]

    def forward(self, x):
        T = x.size(1)
        return x + self.pe[:, :T]   # 임베딩에 그대로 더하기
```

register_buffer로 등록하면 학습 대상 파라미터는 아니지만 model.to(device) 시 함께 이동합니다 — PE는 학습되지 않는 고정값이기 때문.



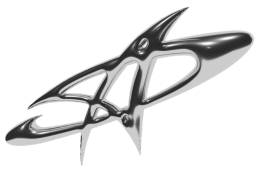
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Positional Encoding 시각화

파형이 실제로 어떻게 생겼는지 확인

- 각 행 = 하나의 위치(pos), 각 열 = 임베딩 차원 — 히트맵으로 보면 물결무늬가 보임
- 차원이 낮을수록(주파수가 높을수록) 빠르게, 높을수록 느리게 진동

```
pe = PositionalEncoding(d_model=64, max_len=100)
plt.imshow(pe.pe[0, :100, :64].numpy(), cmap="RdBu", aspect="auto")
plt.xlabel("embedding dimension"); plt.ylabel("position")
plt.title("Positional Encoding"); plt.colorbar(); plt.show()
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

4주차 정리

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→Multi-Head = 여러 관점을 병렬로 학습

→MultiHead(Q,K,V) = Concat(head_1..h) W^O

→Attention은 순서를 모르므로 Positional Encoding 필요

→PE는 sin/cos으로 만들어 임베딩에 더함

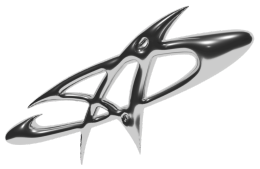
5주차 예고

→Residual Connection과 Layer Normalization

→Position-wise Feed Forward Network

→드디어 하나의 Encoder Block을 완성

→코딩: EncoderLayer → Encoder Stack까지



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지